

How Intent Override Affects Retrieval

Stage S5 — `src/retrieval.py` and `src/state.py`

What Happens When a Customer Changes Their Mind

Here is a concrete example session. The target product is a **full-grain leather belt**.

Turn 1: Customer: "I'm looking for a belt. A key requirement is: canvas material."

Turn 2: Customer: "For that, what matters is: brown color."

Turn 3: Customer: "Actually, ignore my earlier preference. What I need is: full grain leather."

Turn 4: Customer: "For that, what matters is: buckle closure."

On turn 3 the customer changed their mind. They no longer want canvas — they want leather.

What the State Tracker Does (S3)

When the phrase "Actually, ignore" is detected by `OVERRIDE_CUES` in `src/state.py`, `apply_override()` is called. It does **not delete** the old messages. Instead it **down-weights** them to `PRE_OVERRIDE_WEIGHT = 0.35`:

utterances after turn 3:

[turn 1, weight=0.35] "I'm looking for a belt. canvas material."

[turn 2, weight=0.35] "brown color."

[turn 3, weight=1.0] "Actually, ignore my earlier preference. full grain leather."

[turn 4, weight=1.0] "buckle closure."

The state now exposes **two different views** of this history:

View	What it includes	Used for
<code>full_text()</code>	ALL turns (weights ignored, just text concatenated)	Terms route
<code>focused_text()</code>	Only turns where <code>weight >= 1.0</code> (turns 3 and 4 only)	Focused route

```
# src/state.py
```

```
def full_text(self) -> str:
```

```
    return " ".join(utterance.text for utterance in self.utterances)
```

```
    # → "belt canvas material brown color actually ignore full grain leather buckle closure"
```

```
def focused_text(self) -> str:
```

```
    return " ".join(
```

```
        utterance.text for utterance in self.utterances if utterance.weight >= 1.0
```

```
    )
```

```
    # → "actually ignore full grain leather buckle closure"
```

Before any override happens, `focused_text()` and `full_text()` are identical — there is nothing to filter.

What Retrieval Does With Those Two Views (S5)

Only **after an override** does the focused route activate. The check is explicit:

```
# src/retrieval.py

if config.use_terms:
    _rrf(index.search_terms(state.full_text(), ...),
         weight=1.0, sink=fused)           # Route 1: always on

if config.use_focused and state.override_turn is not None:  # Route 2: override only
    _rrf(index.search_terms(state.focused_text(), ...),
         weight=0.8, sink=fused)
```

So on turn 4, **two separate SQL searches** fire against the 50,000-product FTS5 index:

Route 1 — terms (weight = 1.0): all words from all turns

```
MATCH '"belt" OR "canvas" OR "material" OR "brown" OR "color"
      OR "full" OR "grain" OR "leather" OR "buckle" OR "closure"'
```

Route 2 — focused (weight = 0.8): only post-override words

```
MATCH '"full" OR "grain" OR "leather" OR "buckle" OR "closure"'
```

Each route returns its own ranked list of up to 300 products. They are then merged.

Merging the Two Lists: Reciprocal Rank Fusion (RRF)

The two routes produce scores on incompatible scales, so raw scores cannot be added. RRF ignores the scores entirely and uses only **positions**:

`combined_score(product) = sum over routes of: route_weight / (60 + position)`

Consider two candidate products:

Product A — “Full Grain Leather Belt with Buckle Closure”

Route	Position	Score contribution
Terms (weight 1.0)	3	1.0 / (60 + 3) = 0.0159
Focused (weight 0.8)	1	0.8 / (60 + 1) = 0.0131
Total		0.0290

Product B — “Canvas Brown Belt”

Route	Position	Score contribution
Terms (weight 1.0)	5	1.0 / (60 + 5) = 0.0154
Focused (weight 0.8)	— no match —	0.0
Total		0.0154

Product A wins ($0.0290 > 0.0154$) — and it is the correct answer. The focused route gave it an extra boost precisely because it matched the *new* intent while Product B only matched the old, discarded preference.

Why Not Just Delete the Old Words?

The competition specification calls for “slot erasure and rewriting” — when the customer reverses a preference, delete the old one. This implementation **deliberately deviates** from that instruction.

The reason is specific to how the evaluator constructs override sessions. Looking at `behavior_for()` in `evaluator/local_evaluator.py`, the “discarded” preference is **still drawn from the target product’s own metadata**. In the belt example, canvas and brown were genuine attributes of the target product — they were just not the ones the customer ultimately prioritised.

Erasing those words entirely would throw away retrieval signal that the terms route can still exploit. Down-weighting to 0.35 keeps them contributing weakly while giving the focused route a clean, uncontaminated channel carrying the new intent at full strength.

In measured terms: erasing costs score; down-weighting preserves it.

Summary

Before any override:

```
focused_text() == full_text()
→ only the terms route fires
```

After override detected on turn N:

```
full_text()      = all turns 1 ... N ... end   → terms route   (weight 1.0)
focused_text() = turns N ... end only         → focused route (weight 0.8)
→ both routes fire, results merged by RRF
```

Effect on the candidate pool:

```
Products matching the NEW intent → appear in both routes → double boost
Products matching only OLD intent → appear in terms route only → single, weaker score
Products matching neither         → absent from pool
```

The focused route is a clean second opinion: *“ignore everything before the reversal and tell me what is relevant to what the customer actually wants now.”* The terms route stays on so that the pre-override evidence — still derived from the target — is not wasted.